

SERVICIO NACIONAL DE APRENDIZAJE – SENA

Centro de Servicios y Gestión Empresarial · Coordinación de Teleinformática, SENA Regional Antioquia

MATRIZ DE VALIDACIÓN TÉCNICA – PROYECTO INTEGRADOR REACT + FASTAPI

Ficha:	3406211	Programa:	Tecnólogo en Análisis y Desarrollo de Software (código 228118)
Aprendiz:	JUAN DIEGO CARTAGENA TUBERQUIA	Instructor:	César Augusto Moreno Mena
Proyecto:	Catálogo de Motos – BIXE	Fecha de validación:	_____

Autoevaluación del aprendiz. Las casillas «Cumple» y la columna de evidencia las diligencia el aprendiz, indicando dónde se comprueba cada criterio. El resultado general y las firmas quedan en blanco: los diligencia el instructor. Repositorio: <https://github.com/juandicar1298-byte/bixe> · Web: <https://bixe.vercel.app> · API: <https://bixe-api.onrender.com>

Competencia (Norma SofiaPlus)	Resultado de Aprendizaje (RAP)
38367 - Estructurar propuesta técnica de servicio de TI según requisitos técnicos y normativa	593060 - 01 Definir especificaciones técnicas del software de acuerdo con las características del software a construir.
38356 - Implementar la solución de software de acuerdo con los requisitos de operación y modelos de referencia	593109 - 03 Documentar el proceso de implantación de software siguiendo estándares de calidad.
38356 - Implementar la solución de software de acuerdo con los requisitos de operación y modelos de referencia	593112 - 04 Implantar el software de acuerdo con los niveles de servicio establecidos con el cliente.

Criterio verificable	Cumple	No cumple	Observaciones / evidencia
1. Diseño y fundamentos REST			
Los endpoints del dominio están definidos con criterio REST (recurso – verbo HTTP – ruta – código de respuesta).	X		11 routers en app/routers/. Ejemplo: GET /api/productos 200, POST 201, PATCH 200, DELETE 204 y 404 si no existe. Visible en https://bixe-api.onrender.com/docs
Implementa parámetros de ruta y de consulta con validación (Query, Path, Annotated / type hints).	X		Annotated[int, Path(ge=1)] en dependencias.py; Query(ge=0), Literal y min_length en productos.py. Seis pruebas lo comprueban en test_catálogo_crud.py.
El CRUD completo (crear, listar, consultar, actualizar, eliminar) está implementado sobre los recursos principales del dominio.	X		Productos, servicios, usuarios, pedidos, ventas y PQR. Prueba: test_ciclo_de_vida_completo_de_un_producto.
2. Modelado y validación de datos (Pydantic)			
Usa esquemas Pydantic v2 separados para entrada y salida (Create / Update / Response).	X		12 archivos en app/schemas/. Ejemplo: ProductoCrear, ProductoActualizar y ProductoRespuesta son clases distintas.
Incluye validaciones propias del dominio (field_validator / model_validator, restricciones de Field).	X		field_validator: contraseña segura, limpieza de espacios. model_validator: tarjeta no vencida, contraseñas que coinciden, carrito sin artículos repetidos.
3. Persistencia de datos (SQLAlchemy)			
Define al menos dos entidades relacionadas mediante modelos SQLAlchemy 2.0.	X		18 tablas en app/models/bixe.py con Mapped y mapped_column (sintaxis 2.0) y 18 claves foráneas.
El CRUD es persistente contra una base de datos real (SQLite / PostgreSQL), no en memoria.	X		MySQL/MariaDB en local y PostgreSQL (Neon) en producción, con el mismo código. Comprobable en https://bixe-api.onrender.com/api/productos
4. Autenticación y autorización			
Implementa inicio de sesión con JWT (flujo OAuth2 password u equivalente).	X		POST /api/auth/login devuelve el JWT; /api/auth/token atiende el botón «Authorize» de Swagger. 26 pruebas en test_autenticacion.py.
Protege endpoints sensibles según autenticación y/o rol del usuario.	X		ExigirPermiso y ExigirRol en dependencias.py, resueltos contra la tabla roles_permisos y no con ids en el código. Se distingue 401 de 403.
5. Manejo de errores y middlewares			
Maneja errores de forma estandarizada (HTTPException, códigos 404/422, mensajes claros).	X		Jerarquía de excepciones en app/errores.py y manejadores en main.py. Todo error sale igual: {codigo, mensaje, ruta, detalles}.

Criterio verificable	Cumple	No cumple	Observaciones / evidencia
Configura CORS para permitir el consumo de la API desde el frontend en React.	X		CORSMiddleware con lista explícita de orígenes, sin comodín. Comprobado: un origen ajeno no recibe la cabecera Access-Control-Allow-Origin.
6. Asincronía y tareas en segundo plano			
Incluye al menos un endpoint o flujo implementado con async/await.	X		Toda la API es asíncrona sobre SQLAlchemy async (aiomysql en local, asyncpg en Neon). El chatbot espera a Groq sin bloquear el resto de peticiones.
Usa BackgroundTasks (u otro mecanismo equivalente) para al menos una tarea no bloqueante.	X		BackgroundTasks en POST /api/auth/recuperar: el correo sale después de haber respondido, así el usuario no espera los 2–5 segundos que tarda Gmail.
7. Integración de Inteligencia Artificial			
El proyecto expone al menos un endpoint que integra un modelo propio o un servicio de IA externo.	X		POST /api/chat, con Groq (openai/gpt-oss-120b). Estado en https://bixe-api.onrender.com/api/chat/estado . Si no hay clave, responde con el catálogo en lugar de fallar.
Las credenciales o llaves de API se gestionan por variables de entorno, nunca en el código.	X		IA_API_KEY por variable de entorno. El .env está en .gitignore; se versiona .env.example, con los nombres y sin ningún valor.
8. Documentación y preparación para despliegue			
La documentación automática (/docs, /redoc) está personalizada con tags, descripciones y ejemplos.	X		12 etiquetas con descripción y ejemplos en cada esquema. https://bixe-api.onrender.com/docs , /redoc y /openapi.json responden 200.
Incluye README con instrucciones de instalación y ejecución, y requirements.txt / .env.example.	X		README.md con puesta en marcha y despliegue; requirements.txt, requirements-dev.txt y .env.example tanto en el backend como en el frontend.
9. Pruebas (Testing)			
Incluye pruebas con Pytest / TestClient que cubren al menos el CRUD y la autenticación.	X		96 pruebas con Pytest sobre SQLite en memoria (no requiere MySQL ni Neon). Se ejecutan solas en GitHub Actions en cada subida.
10. Comparativa técnica y sustentación			
Incluye un análisis o sección comparativa entre FastAPI y Django REST Framework aplicado al proyecto.	X		docs/10-comparativa-fastapi-django.md. Incluye los tres puntos en los que Django habría sido mejor: panel de administración, migraciones y autenticación.
El aprendiz sustenta con claridad el funcionamiento del proyecto y las decisiones técnicas tomadas.			Se evalúa durante la sustentación.
11. Manual técnico			
Entrega un manual técnico del proyecto: arquitectura, requisitos, instalación, configuración y operación.			Pendiente de entrega por parte del aprendiz.
12. Integración y despliegue continuo (CI/CD)			
Existe un flujo de integración continua en GitHub que revisa el estilo y ejecuta las pruebas automáticamente en cada cambio.	X		.github/workflows/ci.yml: ruff y 96 pruebas en el backend, eslint y compilación en el frontend. Última ejecución en verde: https://github.com/juandicar1298-byte/bixe/actions
El despliegue a la nube es automático y está condicionado a que la integración continua termine correctamente.	X		.github/workflows/despliegue.yml se dispara solo si CI termina en success, y comprueba que https://bixe-api.onrender.com/salud vuelva a responder 200.
13. Pasarela de pago			
El proyecto implementa un flujo de pago completo con validación del medio de pago y emisión del comprobante.	X		Tarjeta, PSE y Nequi en un solo endpoint (unión discriminada). Valida con el algoritmo de Luhn y emite factura consecutiva con IVA discriminado y PDF descargable.
Los datos sensibles del medio de pago no se almacenan (no se guarda el número completo ni el código de seguridad).	X		Solo se guardan la marca y los cuatro últimos dígitos. La tabla pagos no tiene columna para el número ni para el CVV; hay una prueba que lo verifica.
14. Base de datos (normalización)			
El modelo de datos está normalizado hasta la tercera forma normal (1FN, 2FN y 3FN), y las desnormalizaciones están justificadas.	X		docs/14-normalizacion-base-de-datos.md. Las tres desnormalizaciones son deliberadas: las líneas de pedido y de venta congelan nombre y precio porque una factura emitida no puede cambiar de importe.

Criterio verificable	Cumple	No cumple	Observaciones / evidencia
Las relaciones declaran integridad referencial y restricciones de unicidad coherentes con el dominio.	X		18 claves foráneas con CASCADE, SET NULL o restricción según el caso. UNIQUE en correo, documento, consecutivos de factura y de venta, radicado de PQR y referencia de pago.
15. Respuesta a conceptos y principios			
Explica con ejemplos del propio proyecto qué son clase, objeto, herencia y polimorfismo.	X		docs/15-conceptos-y-principios.md, secciones 1 a 5. Herencia: la jerarquía de app/errores.py. Polimorfismo: los tres medios de pago en un solo endpoint.
Explica en qué proceso las clases se convierten en objetos (instanciación: __new__ y el constructor __init__).	X		docs/15-conceptos-y-principios.md, sección 3. Incluye la segunda vía: cómo SQLAlchemy convierte cada fila de la tabla en un objeto del modelo.
Explica la utilidad de cada uno de los componentes del proyecto (capas del backend y carpetas del frontend).	X		docs/15-conceptos-y-principios.md, sección 7: el recorrido de una petición por las capas y para qué sirve cada carpeta.
16. Seguridad: prevención de inyección SQL			
FORMA 1 — Frontend: el inicio de sesión está escalonado en dos formularios, de modo que la contraseña no forma parte de ninguna consulta.	X		frontend/src/components/Login.jsx en dos pasos. Comprobado que el paso 1 no hace ninguna petición al servidor: si preguntara si el correo existe, se podrían enumerar cuentas.
FORMA 2 — Backend: las consultas son preparadas (prepared statement): el valor viaja como parámetro y no dentro de la instrucción SQL.	X		Cero consultas escritas a mano en toda la aplicación. 24 pruebas en test_inyeccion_sql.py, incluida una que compila la consulta contra MySQL, PostgreSQL y SQLite.

Total de criterios evaluados	32	Criterios que cumple (autoevaluación)	30
Resultado general	[] Aprueba [] No aprueba [] Aprueba con observaciones		

Dos criterios quedan sin marcar: el manual técnico, pendiente de entrega, y la sustentación oral, que se evalúa en vivo. Los otros 30 tienen su evidencia indicada arriba.

Observaciones generales:

César Augusto Moreno Mena

Instructor técnico

JUAN DIEGO CARTAGENA TUBERQUIA

Aprendiz